# SLM Report

Sriram Mudunuri Jenny Yan

January 2, 2026

University of Wisconsin–Madison
*Work conducted under the supervision of Prof. Mark Saffman*

## Abstract

*This report describes how to generate optical tweezers and bottle beams, and how to run the accompanying code.*

# Contents


**1 Introduction** **1**

**2 Experimental Setup** **2**

2.1 SLM Setup . . . 2

**3 Background and Theory** **3**

3.0.1 Fourier Transforming property of lens . . . 3

3.0.2 Affine Transformation . . . 3

3.0.3 Code . . . 3

**4 Phase Retrieval Methods** **4**

4.1 Iterative Fourier Transform Algorithms . . . 4

4.1.1 Code . . . 5

4.1.2 Generating a uniform array of tweezers . . . 5

4.2 Gradient Based . . . 7

4.2.1 Code . . . 7

4.2.2 Results . . . 7

4.3 Neural Holography . . . 8

**5 Bottle Beams** **9**

5.1 Generating Bottle Beams: Survey . . . 9

5.2 Code . . . 10

5.3 Results . . . 10

**6 Modulated Bottle Beams** **10**

6.1 Analytic Phase masks . . . 11

6.1.1 Phase Mask 1 . . . 12

6.1.2 Phase Mask 2 . . . 12

6.1.3 code . . . 13

**7 Other results** **13**

7.1 Fast readout traps . . . 13

7.2 Interleaved bottles and tweezers . . . 13

7.3 Optimal transport 2d bottle beam . . . 15

7.4 Code . . . 15

**8 Aberration Correction** **16**

8.1 Super pixel . . . 16

8.1.1 Code & Results . . . 16

8.2 Zernike . . . 16

8.2.1 Iterative Zernike perturbation (IZP) . . . 17

8.2.2 Bayesian Optimization in Zernike Kernel Space . . . 19

8.2.3 Code . . . 22

**9 Quadratic Bottle Beams** **23**
9.0.1 Code . . . 24

**10 Outlook and Future Improvements** **25**
10.1 Tweezers . . . 25
10.2 Aberration Correction . . . 25
10.3 3d optimisation . . . 25

**11 Lab Manual (Code Usage)** **25**
11.1 Repository and branch structure . . . 25
11.2 Recommended workflow . . . 26
11.3 Local setup . . . 26
11.3.1 Clone and checkout . . . 26
11.3.2 Python dependencies . . . 26
11.3.3 Path convention used by the repo . . . 26
11.4 Module map (what each file is for) . . . 27
11.4.1 `master` branch (lab workflow) . . . 27
11.4.2 `simulation` branch (simulation workflow) . . . 27
11.5 Minimal run recipes (copy/paste) . . . 27
11.5.1 (A) Lab workflow (`master`) . . . 27
11.5.2 Tweezer array generation (using our GS / UltraPrecise code) . . . 28
11.5.3 (B) Simulation workflow (`simulation`) . . . 29
11.6 Practical notes . . . 30

**12 References** **31**

# 1 Introduction

Neutral atoms offer a promising future for quantum computing because of their long coherence times and scalability. One of the main challenges they face is performing high-fidelity mid-circuit measurements without disturbing the rest of the quantum register. A recent proposal [1] suggests using two species of atoms, rubidium (Rb) and caesium (Cs), to solve this. The idea is to use one species as the main qubit register and the other as ancilla qubits, which can be measured mid-circuit for feedback or error correction without affecting the computational qubits.

To actually implement this, we need to trap both species in the same region of space and be able to manipulate them individually. Different atoms have different polarisabilities at a given wavelength, meaning they experience different optical dipole forces. A trap that confines one species might repel the other, or trap them with very different strengths. To get around this, a combination of bright and dark traps are used. Bright traps (from red-detuned light) attract atoms to high-intensity regions, while dark traps (from blue-detuned light) repel them from high-intensity regions.

To create and control these light fields, we use Spatial Light Modulators (SLMs), which allow us to program arbitrary phase patterns onto a laser beam. By calculating and uploading the right phase map, we can sculpt the beam into any intensity pattern we want, including arrays of bright and dark traps. SLMs are especially useful because they're fully reconfigurable: we can test and generate many trap patterns without physically changing the optics. This makes them ideal for dual-species neutral atom experiments. While bright traps using SLMs have been widely demonstrated [2], generating and combining dark traps within the same programmable system remains relatively unexplored, and is the main focus of this work.

## 2 Experimental Setup

The experimental setup consists of a Spatial Light Modulator (SLM), Beam expander, Camera, Beam Splitter. The SLM is programmed with a phase map that is computed based on the desired trap intensity pattern, assuming ideal optics. However, in practice, environmental disturbances, optical misalignment, and imperfections in the SLM introduce aberrations that distort the resulting intensity distribution. These effects are mitigated through a feedback error correction loop that re-adjusts the phase map until the measured intensity closely matches the desired distribution.

In our experiments, we assess trap performance by measuring its uniformity. Ensuring each site has the same trapping depth is crucial for consistent qubit control. We quantify uniformity from trap images by extracting the peak intensity, beam waist (in both x and y), and center position for every site, and then evaluating the variation across the array.

### 2.1 SLM Setup

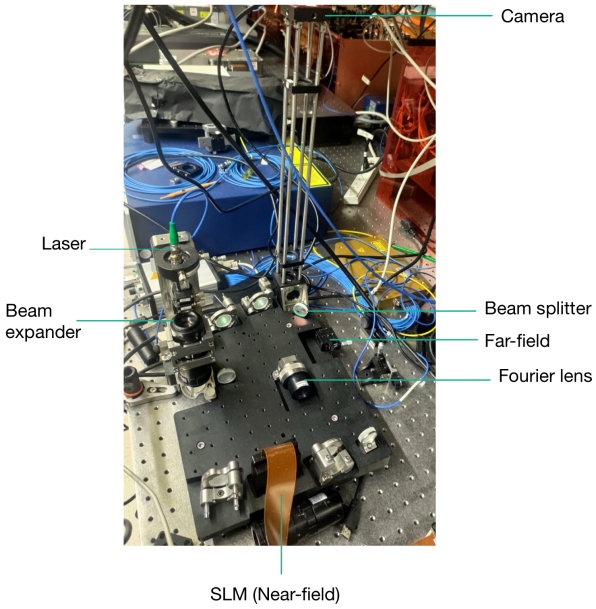



Figure 1: Experimental setup in the lab.

A phase only Spatial Light Modulator has the ability to arbitrarily control the phase of the reflected light. In the setup shown in Fig. 1 a collimated gaussian beam (laser of wavelength 1064nm) is incident on a SLM, This area of SLM is referred as near field. The reflected light is passed through a lens which is one focal length away from the SLM. An image forms at the other focus (also referred to as far-field). The far-field light is split using a beam splitter and passed to the camera for imaging and the other split is passed to the Trapping area. A 4f lens setup is used to refocus the far-field image to the camera. By controlling the phase of the SLM we can control the distribution of light at the far-field a process called as Computer Generated Holography.

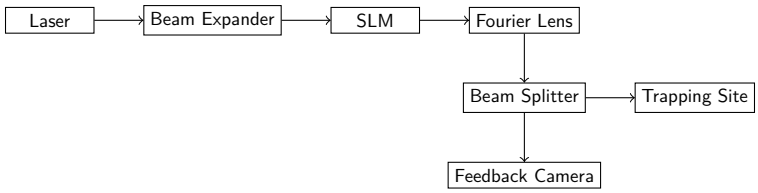



Figure 2: Block diagram of the SLM optical setup. The feedback camera receives a portion of the beam after the splitter and the other part goes to the far-field.

# 3 Background and Theory

### 3.0.1 Fourier Transforming property of lens

Under Fresnel diffraction, a lens performs a Fourier transform of the complex amplitude at front focal plane ($U_{nf}$) to back focal plane ($U_{ff}$).

$$U_{ff}(x_2, y_2) = -\frac{ik}{f}\,\mathcal{F}\{U_{nf}(x_1, y_1)\} = -\frac{ik}{2\pi f}\iint U_{nf}(x_1, y_1)\, e^{\frac{ik}{f}(x_1x_2+y_1y_2)}\, dx_1\, dy_1. \tag{1}$$

where $U(x, y) = A(x, y)e^{i\phi(x,y)}$.

The integral in Eq. (1) can be generalized to account for focus control along the axial ($z$) direction by incorporating a quadratic lens phase term at the input:

$$U_{ff}(x_2, y_2; z_2) = -\frac{ik}{2\pi f}\iint A_{nf}(x_1, y_1)\, e^{i\phi_{nf}(x_1,y_1)} e^{\frac{ikz_2}{2f^2}(x_1^2+y_1^2)} e^{\frac{ik}{f}(x_1x_2+y_1y_2)}\, dx_1\, dy_1. \tag{2}$$

In practice, we work with a discretized version of this propagation, given by:

$$U_{ff}[m, n] = \sum_{i,j} A_{nf}[i, j]\, e^{i\phi_{nf}[i,j]} e^{\frac{ik}{2f_l}(x_i^2+y_j^2)} e^{\frac{ikz_2}{2f^2}(x_ix_m+y_jy_n)}\, \Delta x\, \Delta y. \tag{3}$$

where $(i, j)$ and $(m, n)$ index the discrete spatial coordinates at the input and output planes respectively, and $\Delta x$, $\Delta y$ are the spatial sampling intervals.

### 3.0.2 Affine Transformation

The camera captures a scaled version of the far-field image depending on the 4f lens setup between the far-field and the camera. Depending on the alignment, there could be a rotation too. So the amplitudes between Camera and Far Field can be related by an affine transformation.

$$\begin{aligned} A_{cam}(x_3, y_3) &= A_{ff}(x_2, y_2) \\ \text{where} \quad \begin{bmatrix} x_3 \\ y_3 \end{bmatrix} &= \mathbf{M} \begin{bmatrix} x_2 \\ y_2 \end{bmatrix} + \mathbf{t} \end{aligned} \tag{4}$$

### 3.0.3 Code

We used the fast fourier transform implemented in scipy and pytorch. For the affine transformation we used the slmsuite's fourier calibration method.

# 4 Phase Retrieval Methods

The phase retrieval problem deals with estimating the near-field phase which creates the desired hologram. Here the far-field phase is a degree of freedom we have.

Given a target amplitude $A_{\text{target}}(x_3, y_3)$, the **phase retrieval problem** is:

$$\boxed{\begin{aligned}&\text{Find } \phi_{nf}(x_1, y_1) \text{ such that} \quad A_{\text{cam}}(x_3, y_3) \approx A_{\text{target}}(x_3, y_3),\\ &\qquad \text{given } A_{nf}(x_1, y_1) \text{ and subject to} \quad \text{Eqs. (1) and (2).}\end{aligned}}$$

The Algorithms to solve for $\phi_{nf}$ can be characterised to three types. We discuss each of these in detail.

## 4.1 Iterative Fourier Transform Algorithms

Gerchberg & Saxton [3] came up with an iterative fourier transform algorithm (GS) to solve the phase retrieval problem and proved its convergence. A general IFTA looks like

**Algorithm 1** IFTA

**Require:** Known input amplitude $A_{\text{input}}$, target output amplitude $A_{\text{target}}$, number of iterations $T$

**Ensure:** Estimated input phase $\phi_{nf}$

1: Initialize $\phi^{(0)}$ randomly
2: **for** $t = 1$ to $T$ **do**
3: $\quad U_{nf}^{(t)} \leftarrow A_{\text{input}} \cdot e^{i\phi^{(t-1)}}$
4: $\quad U_{ff}^{(t)} \leftarrow \mathcal{F}[U_{nf}^{(t)}]$
5: $\quad U_{ff}^{(t)} \leftarrow A_{\text{target}} \cdot \frac{U_{ff}^{(t)}}{|U_{ff}^{(t)}|}$
6: $\quad U_{nf}^{(t)} \leftarrow \mathcal{F}^{-1}[U_{ff}^{(t)}]$
7: $\quad U_{nf}^{(t)} \leftarrow A_{\text{input}} \cdot \frac{U_{nf}^{(t)}}{|U_{nf}^{(t)}|}$
8: $\quad \phi^{(t)} \leftarrow \arg\left(U_{nf}^{(t)}\right)$
9: **end for**
10: **return** $\phi_{nf} \leftarrow \phi^{(T)}$

$A_{input}$ is assumed to be uniform in Simulation.For tweezer arrays Line 4 in the algorithm becomes as shown in [2]:

$$U_{ff}^{(t)} = \arg\left(\sum_m \frac{w_m^t U_{nf}^t[m]}{|U_{nf}^t[m]|}\right) \tag{5}$$

where m denotes the tweezer index. The weights $w_m^t$ are 1 for GS Algorithm. The tweezer arrays created using this suffers from lack of uniformity. [4] improved the uniformity by modifying the target weights based on camera feedback $w_m^{(t)} = w_m^{(t-1)} \cdot \frac{\langle |A_{\text{cam}}^{(t-1)}[m]| \rangle}{|A_{\text{cam}}^{(t-1)}[m]|}$ knows as Weighted GS Algorithm . This algorithm optimizes both weights and phases at the same time, [5] further showed that fixing the phases for tweezers after reaching sufficient iterations gives better results in experiments.

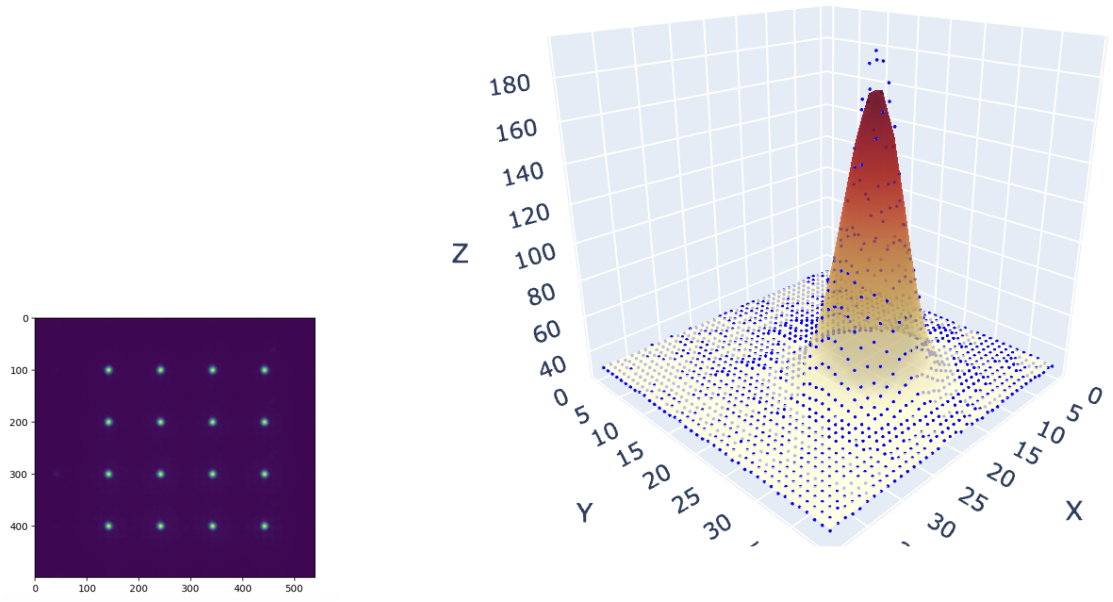
One more issue the tweezers face is shape inhomogeneity.[6] solved this issue using a technique of optical apodisation where they modify $w_m^{(t)}(x_1, y_1) = a_m \exp\left(-\frac{x_1^2}{X_m^2} - \frac{y_1^2}{Y_m^2}\right)$ and optimize for $X_m$ and $Y_m$ and achieved the most homogeneous spots till date. Other approaches for tweezers include [7] with exact solutions for specific geometries and MRAF algorithm [8] which separates the far-field into two regions signal and noise and exploits the phase degree of freedom in noise region to get smooth holograms.[4] discusses various other aprroaches like Random Superpostion (RS),S etc. which lacks at either uniformity or efficiency.

### 4.1.1 Code

The slmsuite has implemented for gerchberg saxton (gs) and weighted gerchberg saxton (wgs) algorithms in scipy. We implemented a faster version of these in pytorch for tweezers. We also implemented the optical apodisation [6] to correct for uniformities. These can be found in src/algos/gs_spots,ultraprecise_spots.

### 4.1.2 Generating a uniform array of tweezers

We used the GS algorithm for 50 iterations to create a set of tweezers shown in Fig. 3. The target Amplitude $A_{target}$ for the algorithm is a superposition of delta functions at the coordinates of the spots and the input amplitude $A_{nf}$ is assumed to be uniform. This works because the input amplitude is a gaussian beam and its far-field amplitude is a diffraction limited gaussian spot. As per Fourier Convolution theorem. The far field delta functions convolve with the gaussian spot thus creating the array of tweezers.




(a) Tweezer array

(b) 3D fit of a single tweezer

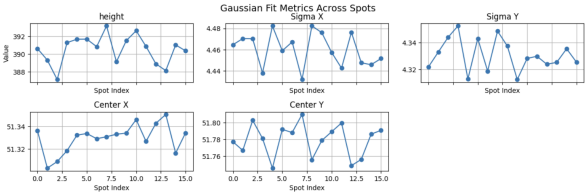
Figure 3: Optical tweezers generated using an SLM.

We improved the Amplitude and waist uniformity of tweezers using the iterative method

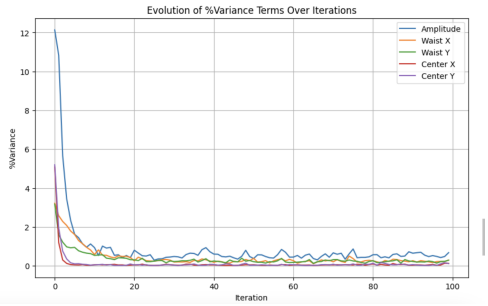
as in [6] and discussed in subsection 4.1. For correcting uniformity of X,Y positions we added Zernike polynomials $Z_1^1$, $Z_1^{-1}$ iteratively at a spot level. The results and convergence of this method are shown in Fig. 4.The uniformity is calculated by finding the mean difference between the uniformity parameters over each site of the grid. We calculated uniformity of intensity using

$$\mathcal{U} = \frac{I_{\max} - I_{\min}}{I_{\max} + I_{\min}} \tag{6}$$

We found Amplitude uniformity of 99.3%, height uniformity of Waist in X : 99.4% , Waist in Y : 99.5%, X position : 99.95% and Y position : 99.95%




(a) Metrics across tweezers




(b) Convergence of variances over iterations

Figure 4: Uniformity of Tweezers.

### 4.2 Gradient Based

Gradient descent methods solve the phase retrieval problem by iteratively updating the near-field phase $\phi_{nf}(x, y)$ to minimize a cost function, typically defined as the difference between the desired target amplitude $A_{target}(x, y)$ and the generated amplitude $A_{cam}(x, y)$. Specifically, the phase update follows the gradient of a cost function $C(\phi)$ as:

$$\phi_{nf}^{t+1}(x, y) = \phi_{nf}^{t}(x, y) - \eta \frac{\partial C(\phi)}{\partial \phi_{nf}(x, y)}, \tag{7}$$

where $\eta$ is the step size. Chandra et al. [9] developed PhasePack, which offers various gradient-based algorithms such as Wirtinger Flow and gradient descent, efficiently handling practical phase retrieval tasks with various initialization methods.

Non-convex optimization has been further explored by Sui et al. [10] and Zhang et al. [11], who utilized specifically designed cost functions and iterative optimization schemes to significantly enhance hologram fidelity for complex 3D intensity patterns. In addition, conjugate gradient methods introduced by Harte et al. [12] optimize holograms by updating the phase as:

$$\phi_{nf}^{t+1}(x, y) = \phi_{nf}^{t}(x, y) + \alpha_t d_t(x, y), \tag{8}$$

where $d_t(x, y)$ is a conjugate direction derived from gradients in previous iterations, and $\alpha_t$ is the optimal step size determined by line search. These approaches are shown to provide precise control over optical tweezers in simulations.

#### 4.2.1 Code

We implemented gradient based methods for both 2D and 3D phase retrieval in pytorch at src/gradient.py . We used the torchmin package for the minimizers for optimization which support various minimizers like conjugate gradient, lbfgs etc. The class SLMphaseRetrievalCG is modular; we implemented support for padding the input SLM which can create the output holograms at a finer resolution than default in simulation. This is especially useful for diffraction limited bottle beams.

We also have the development notebook src/notebooks/gradient.ipynb and src/notebooks/report.ipynb with code in testing demonstrating the conjugate gradied method for 2d and 3d holograms.

#### 4.2.2 Results

A modulated bottle beam generated using conjugate gradient method is shown in Fig. 5. Since it is generated using 2d gradient descent method it does not have a 3d trapping profile. We discuss more about bottle beams in section 5. The results tell that analytically designed phase masks are better suited for generating diffraction limited bottle beams than gradient based methods which gave non uniform lobes as in 5b .




(a) 2D heat map

(b) 2D radial profile plotted in 3d

Figure 5: Modulated Bottle Beam generated using 2D Conjugate Gradient method.

### 4.3 Neural Holography

According to equation (1), the amplitude of the far-field can be written as a function of the SLM phase,

$$A_{ff} = f(\phi_{nf})\,.$$

Ideally, $f$ is the modulus of a linear DFT of the complex exponential of the input phase. In the presence of aberrations, the relationship becomes more complicated, but one can model $f$ as a parameterized neural network. Training this network on $\{\phi_{nf}, A_{cam}\}$ pairs collected from the SLM and camera, we obtain an approximate differentiable model $\hat{f}$ of the SLM setup. We can then apply the previously discussed gradient methods to find the SLM phase for any desired target hologram. [13] demonstrated how to train such a model (Holonet) with camera feedback using the CITL method and obtained promising results. [14] proposed a similar CITL approach for training a CNN model, treating $A_{ff}$ as the parameters to optimize instead of $\phi_{nf}$.

An alternative is to train a model that learns a function $\hat{g}$ which predicts $\phi_{nf}$ given $A_{\text{target}}$, i.e. $\hat{g}$ approximates

$$\phi_{nf} = g(A_{\text{target}})\,.$$

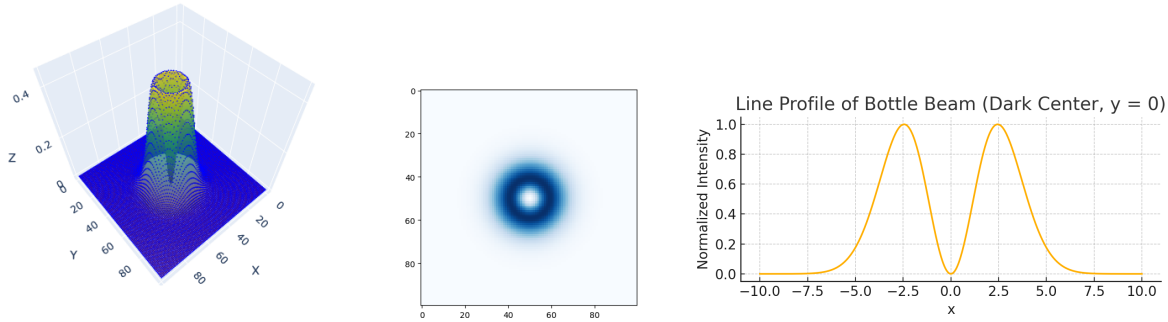
[15] demonstrated this using GANs to create optical tweezers. The advantage of this approach over Holonet is that it does not rely on any iterative or optimization algorithm, allowing optical tweezers to be created or aligned much faster—potentially enabling faster qubit gates and measurements.

Other uses of deep learning in CGH include neural-network-based tweezer controllers [16]. [17] provides a comprehensive review of integrating deep learning into tweezer design, control, calibration, and trapping.

# 5 Bottle Beams

A bottle beam, also known as a blue-detuned optical trap, has a dark center surrounded by a bright region (Fig. 6) and can be used to trap atoms. Multiple types of bottle beams include Gaussian-interference and vortex beams. Gaussian-interference bottle beams are described in [18]. In this approach, two Gaussian beams of different waists interfere destructively to create a dark region at the center, as shown below.

$$A_{ff}(x,y) = A_1 \exp\left(-\frac{x^2+y^2}{w_1^2}\right) - A_2 \exp\left(-\frac{x^2+y^2}{w_2^2}\right) \tag{9}$$




(a) 3D intensity (b) 2D heat map (c) Line profile

Figure 6: Simulated bottle-beam intensity visualizations.

## 5.1 Generating Bottle Beams: Survey

The most common way to generate a bottle beam is to imprint a 0–$\pi$ phase mask [19], which applies a $\pi$ phase shift to part of the incident laser beam, thus creating two Gaussian beams that destructively interfere. Xiao et al. [20] proposed a gradient-descent method to optimize a bottle beam using a metasurface. They introduced a figure of merit

$$\eta = \frac{I_{\text{escape}} - I_0}{P \cdot 10^{-9}\,\text{m}^2},$$

where $I_{\text{escape}}$ is the lowest peak intensity among all directions, $I_0$ is the intensity at the trap center, and $P$ is the incident power.

Abacousnac et al. [21] used the Zernike phase-contrast principle to create a bottle beam by making a phase-only Zernike approximation to the complex hologram. Zhang et al. [22] demonstrated phase engineering to generate self-bending acoustic bottle beams using sound waves instead of light. Konzett et al. [23] axially shifted two optical tweezers and interfered them using random mask encoding on an SLM [24] to produce a bottle beam. Pérez et al. [25] used two SLMs to improve the uniformity of a vector bottle beam and compared various generation methods in Fig. 3 of their article.

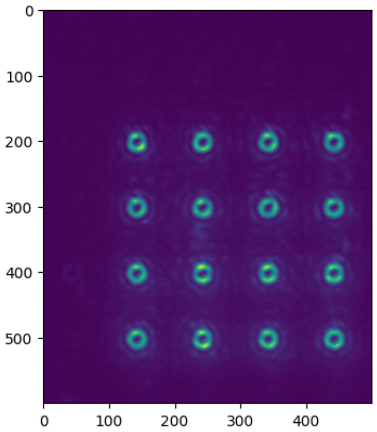
We could create an array of bottle beams by applying the 0–$\pi$ phase mask on top of the tweezer array phase mask discussed in Section 4.1.

## 5.2 Code

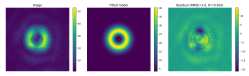
The code to generate pi phase mask is implemented in src/utils/bottle_beam_phase. The code to calculate the radius for the pi phase mask is implemented in src/utils/get_bb_optimal_a followed from konzett et al. We also implemented the gradient based bottle beam generation discussed in section 4.2. bottle.ipynb has a notebook demonstrating the bottle beam generation using conjugate gradient method. The code to fit the bottle beam and make line ,2d and 3d plots in src/plots.py

## 5.3 Results

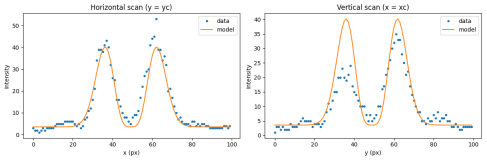
The tweezers in 3a are upgraded to bottle beams by adding the pi phase mask on top of the tweezer phase mask. The results are shown in Fig. 7.



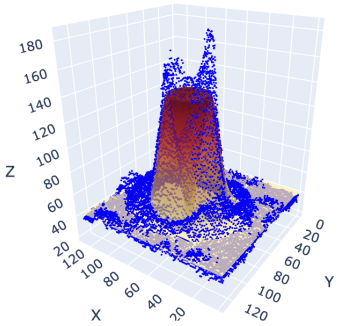

(a) Bottle Beam array



(b) 2D fit of a single bottle beam




(c) Line profile fit of a single bottle beam




(d) 2d fit of a single bottle beam 3d plot

Figure 7: Optical bottle beams generated using an SLM.

# 6 Modulated Bottle Beams

One type of trap that can be used for dual-species trapping is a modulated bottle beam, which is made by modulating the bottle beam ring with a sinusoidal pattern. This creates multiple

bright regions around the bottle beam, allowing for trapping of different atomic species in separate locations. The modulation Fig.8 can be described mathematically as:

$$I(r,\theta) \sim r^2\, e^{-2(r-r_0)^2/w^2}\Big[a + b\cos^2(n\theta)\Big]$$

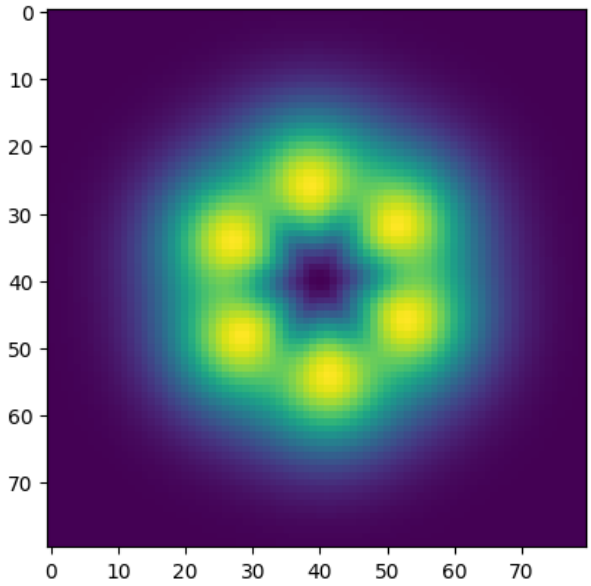



Figure 8: Simulated Modulated Bottle Beam with 6 lobes

We could create them numerically using IFTA or gradient based methods discussed earlier but they don't have 3d trapping profile and 3d gradient algorithms are computationally expensive and a solution is not guaranteed to exist. Further, we don't have an expected plausible 3d profile. So we explored analytic phase masks to generate modulated bottle beams.

### 6.1 Analytic Phase masks

The analytic approach follows from the multipole series expansion of the phasor

$$e^{i\Phi(r,\theta)} \;=\; \sum_{n=-\infty}^{\infty} \varphi_n(r)\; e^{i\,n\,\theta},$$

Now consider the fourier transform of these terms

$$E(k=0,z) = \sum_{n=-\infty}^{\infty} i^n \int_0^{\infty} \varphi_n(r)\, J_n(0)\, \exp\Big(-i\frac{k\,r^2}{2f}\Big)\, r\, dr \;=\; \int_0^{\infty} \varphi_0(r)\, \exp\Big(-i\frac{k\,r^2}{2f}\Big)\, r\, dr.$$

We observe that the fourier transform of phase = n $\theta$ is the hankel transform of the radial part with phase n $\theta$. Thus if phase n $\theta$ is retained in the far-field amplitude. If we observe the modulated bottle beam intensity profile we see that it is combination of n=0 and n = 2n terms. So we could use random superposition method [24] to create a modulated bottle beam pattern by superposing the phase masks for n=0 and n=2n phase terms with weights a and b respectively.

Below, I show two phase masks and simulated far-field intensity profiles.

### 6.1.1 Phase Mask 1

We choose between a bottle beam phase and a $\pm 3\theta$ phase by two independent draws, $rand_1$ and $rand_2$.

$$\phi(r,\theta) = \begin{cases} \phi_{\text{bottle}}(r,\theta), & \text{if } \text{rand}_1 < 0.5, \\ \begin{cases} 3\theta, & \text{if } \text{rand}_2 < 0.5, \\ -3\theta, & \text{otherwise}, \end{cases} & \text{otherwise.} \end{cases}$$

Far Field Amplitude :

Now we superpose the usual $J_{n+1}(r)\cos(2n\theta)$ term with the bottle beam amplitude $E_{\text{bottle}}(r,\theta)$:

$$E_{\text{ff}}(r,\theta) = \left[J_4(r)\ \cos(2n\theta) + E_{\text{bottle}}(r,\theta)\right]\ *\ E_{\text{tweezer}}(r,\theta)$$

Here the input beam which is a gaussian is convolved with fourier trasform of the phasors to get the final intensity distribution.

The simulated far-field intensity profile is shown in Fig. 9.

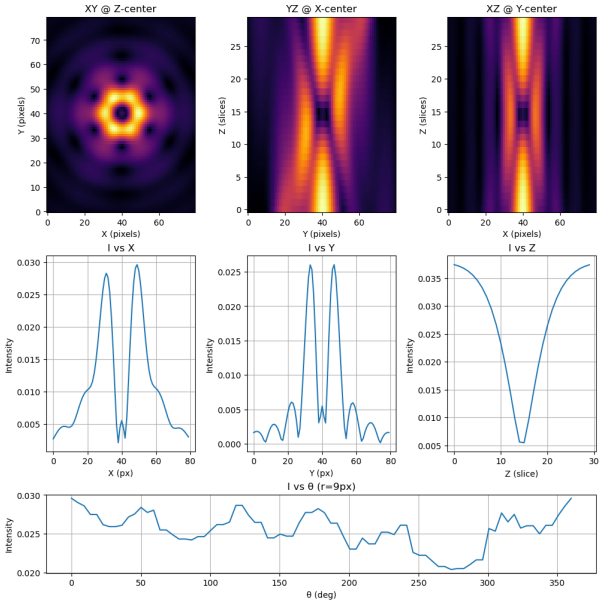



Figure 9: Simulated Modulated Bottle Beam using Phase Mask 1

### 6.1.2 Phase Mask 2

Choose between the simple $6\theta$ phase or the bottle beam phase via one draw $\text{rand}_1$:

$$\phi(r,\theta) = \begin{cases} 6\theta, & \text{if } \text{rand}_1 < 0.5, \\ \phi_{\text{bottle}}(r,\theta), & \text{otherwise.} \end{cases}$$

Far field Amplitude:

Superpose the Bessel term with the bottle beam amplitude:

$$E_{\text{ff}}(r,\theta) = \left[J_7(r)\,\exp(i\,n\theta) + E_{\text{bottle}}(r)\right] \;*\; E_{\text{tweezer}}(r,\theta)$$

The simulated far-field intensity profile is shown in Fig. 10.

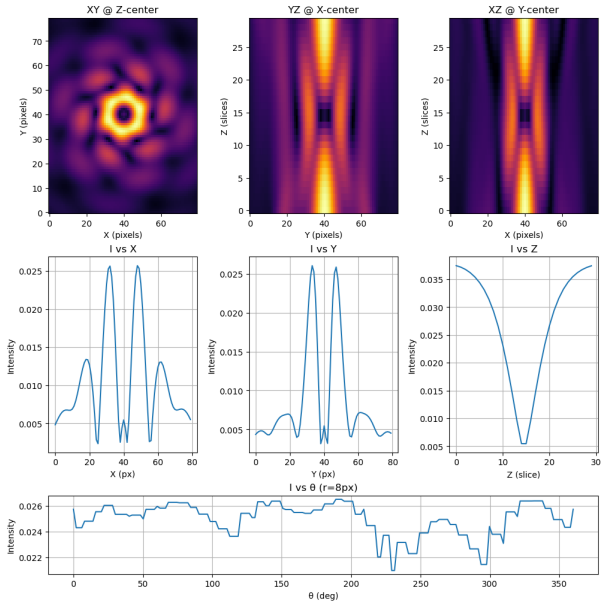



Figure 10: Simulated Modulated Bottle Beam using Phase Mask 2

### 6.1.3 code

The code and results for modulated bottle beams can be found in src/reports/mbb.ipynb.

# 7 Other results

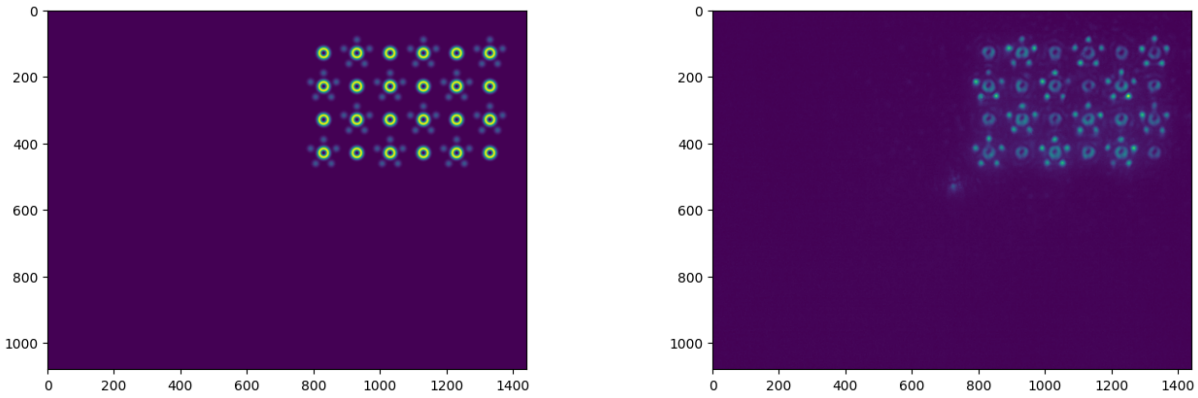
add a section in appendix with put all 2d bottles,periodic bottles by gradient and optimal transport and mention that they don't have 3d profile adding pictures. and add ideas to mitigate these.

## 7.1 Fast readout traps

We upgraded some optical tweezers to bottles to create fast readout traps to implement surface code shown in Fig. 11.

## 7.2 Interleaved bottles and tweezers

We created interleaved bottles and tweezers using the phase mask random superposition method discussed in section 6.2 by superposing array of bottles with array of tweezers shown in Fig. 12.




(a) Simulated Fast Readout Traps

(b) Experimental Fast Readout Traps

Figure 11: Fast Readout Traps for Surface Code Implementation.

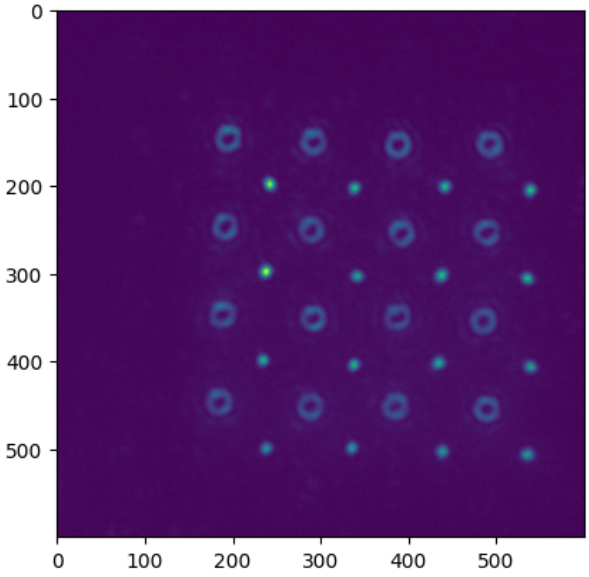



Figure 12: Interleaved Bottles and Tweezers

## 7.3 Optimal transport 2d bottle beam

We tried optimal transport algorithm presented in [26] which showed state of the art results for generating 2d holograms. We created a 2d bottle beam using this method shown in Fig. 13. It doesn't have a 3d trapping profile. The comparison of various algorithms in the paper for bottle beams of different radius is shown in Fig. 14(c).

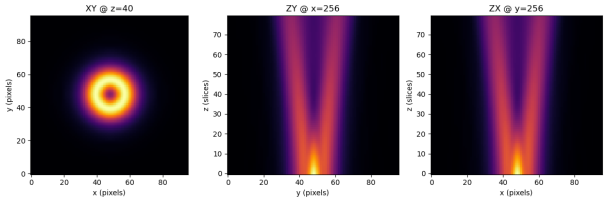



Figure 13: 2D Bottle Beam using Optimal Transport

radius = 1

| method | rms error | efficiency |
|---|---|---|
| Φgs | 0.0168059 | 0.999944 |
| Φmraf | 0.0369136 | 0.654717 |
| Φot | 0.255251 | 1 |
| Φotgs | 0.0832378 | 1 |
| Φotmraf | 0.011659 | 0.693584 |

radius = 2

| method | rms error | efficiency |
|---|---|---|
| Φgs | 0.138175 | 0.999803 |
| Φmraf | 0.0432999 | 0.759891 |
| Φot | 0.349872 | 1 |
| Φotgs | 0.0837408 | 1 |
| Φotmraf | 0.0154087 | 0.775889 |

radius = 4

| method | rms error | efficiency |
|---|---|---|
| Φgs | 0.163687 | 0.999207 |
| Φmraf | 0.0942454 | 0.75946 |
| Φot | 0.115987 | 1 |
| Φotgs | 0.053186 | 0.999992 |
| Φotmraf | 0.00396279 | 0.844005 |

radius = 8

| method | rms error | efficiency |
|---|---|---|
| Φgs | 0.169929 | 0.996431 |
| Φmraf | 0.0870453 | 0.755749 |
| Φot | 0.0478456 | 0.999999 |
| Φotgs | 0.032622 | 0.999949 |
| Φotmraf | 0.000822837 | 0.851998 |

Figure 14: Comparison of various algorithms for generating 2D bottle beams of different radius. Here radius is wrt the waist of diffraction limited tweezer

## 7.4 Code

The random masking code to generate interleaved bottles and tweezers can be found in src/utils/random_maskin
A notebook to demonstrate the fast readout traps is at src/notebooks/surfacecode.ipynb .
The code for optical transport is provided by the author in Julia can be found at Holography/SLMTools. I ported some of it to python at src/pot.py.

# 8 Aberration Correction

## 8.1 Super pixel

One way to correct aberrations globally is find a phase mask to compensate aberrations at a superpixel level described in [27]. Instead of assigning a different phase to each individual pixel, we group pixels into small blocks called superpixels and assign a single phase value to the entire block. We interfere two of these superpixels (one of them is a reference superpixel) at a calibration point and find out the relative phase to be added to correct the aberration. This smooths out local pixel noise and averages over small defects or non-uniformities. It also avoids phase discontinuities that come from trying to display sharp features on a discretized grid.

If the phase assigned to a superpixel is $\phi_{ij}$ and the region it covers is $S_{ij}$, then the resulting field can be written as:

$$U(x,y) = \sum_{i,j} A_{ij}(x,y) \cdot e^{i\phi_{ij}}, \quad \text{for all } (x,y) \in S_{ij} \tag{10}$$

### 8.1.1 Code & Results

We used the slmsuite's wavefront_calibrate function to implement superpixel aberration correction and tuned parameters for our device. The code can be found in src/utils/sp_calibrate . This method also approximates us the incident beam profile on the slm. The results are shown in Fig. 15 and Fig. 18(d) shows the comparison of superpixel aberration correction with other methods discussed later.

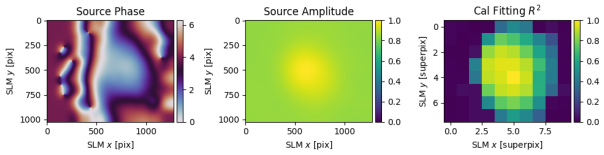



Figure 15: Incident beam Amplitude and phase profile from superpixel aberration correction

## 8.2 Zernike

Zernike polynomials are a sequence of orthogonal polynomials on the unit disk. They are used as basis functions to quantify aberrations. Using the ansi index to order them $Z_n$ represents the nth Zernike polynomial.

Any phase mask on a unit disk can be represented as a linear combination of Zernike polynomials.

$$\phi(r,\theta) \;=\; \sum_{n=0}^{\infty} w_n \, Z_n(r,\theta). \tag{11}$$

where $w_n$ are the weights for each Zernike polynomial called Zernike coefficients. They are in units of radians of phase and can be converted to units of wavelengths.

We discuss two techniques to correct for aberrations using Zernike polynomials. Both techniques approximate the aberration phase profile for a tweezer or bottle beam using a linear

combination of $n_{max}$ Zernike polynomials.

$$\phi_{aberr,i}(r,\theta) \;=\; \sum_{n=0}^{n_{max}} w_{n,i}\, Z_n(r,\theta). \tag{12}$$

### 8.2.1 Iterative Zernike perturbation (IZP)

In this technique, we iteratively measure the optimal Zernike coefficients $w_{n,i}$ by perturbing each Zernike polynomial and measuring the Strehl ratio/ $R^2$/RMSE and correcting the aberration using Gerchberg Saxton Algorithm in Zernike Kernel Space. This is followed from [28]

**Algorithm 2** Iterative Zernike Perturbation

1: **for** $n = 0$ to $n_{\max}$ **do**
2: Step 1: Add $w_n \cdot Z_n$ to $\phi_{\text{tweezers}}$ for $w_n \in [-1, 1]$ in steps of 0.05 (20 steps).
3: Step 2: For each tweezer $(i)$ compute Strehl ratio $(S_{i,w_n})$ for each $w_n$.
4: Step 3: For each tweezer Fit a parabola to Strehl ratio vs. $w_n$ and find its maximum.
5: Step 4: Use Gerchberg–Saxton algorithm in Zernike kernel space to compensate the aberration.
6: **end for**

We repeat the above procedure for 4-5 iterations.

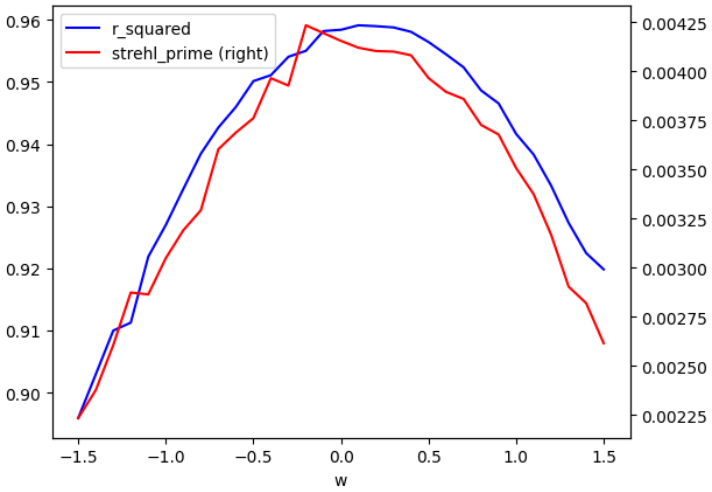



Figure 16: Strehl ratio vs Zernike coefficient perturbation for a single Zernike mode. A parabola is fitted to find the optimal Zernike coefficient that maximizes the Strehl ratio.

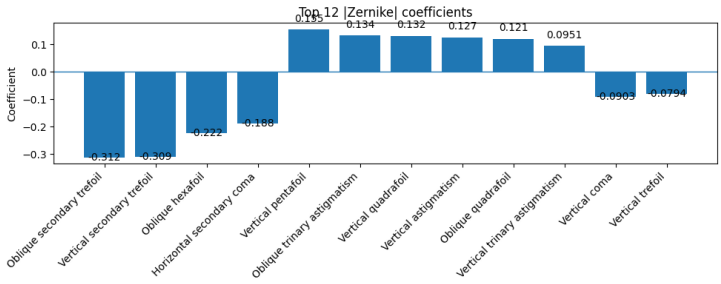
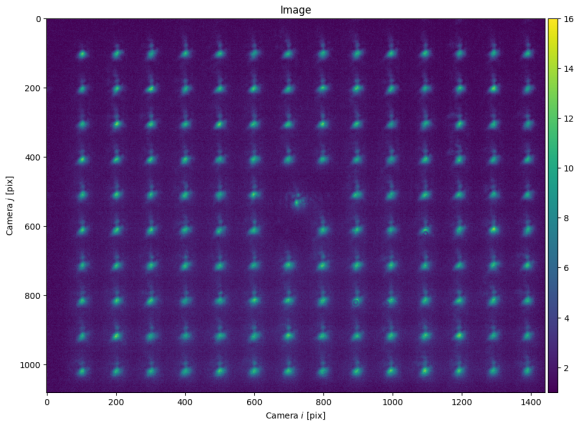
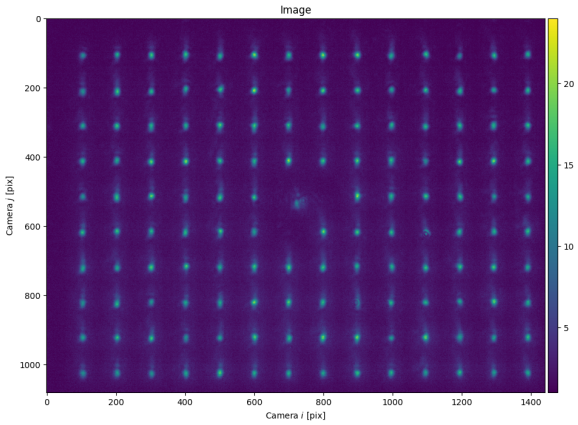


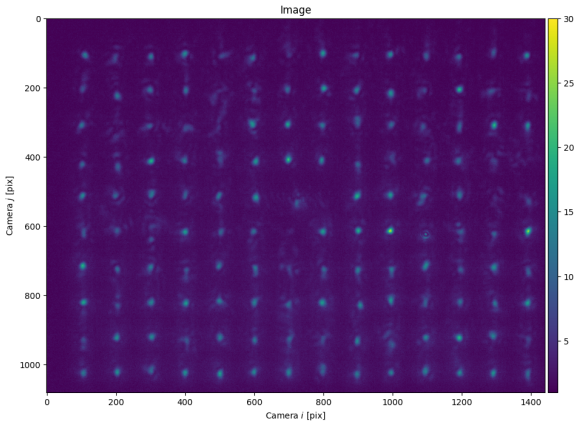

Figure 17: Optimal Zernike coefficients found by iterating the Zernike's. The primary aberrations in the system are shown.



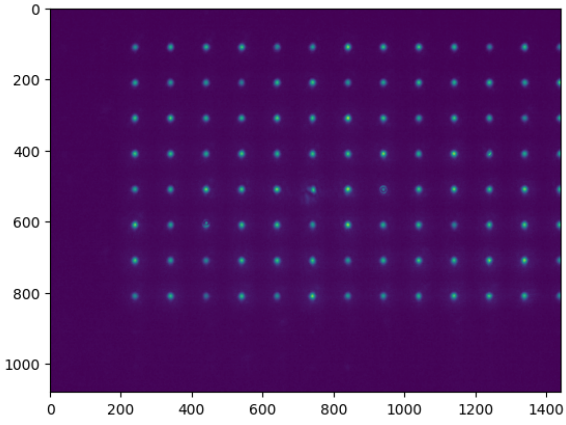

(a) Without aberration correction.




(b) IZP with 9 Zernike terms.




(c) IZP with 25 Zernike terms.



(d) Superpixel aberration correction.

Figure 18: Comparison of aberration correction results with different methods. (a) No correction, (b) IZP with 9 Zernike terms, (c) IZP with 25 Zernike terms, and (d) superpixel aberration correction. The aberration correction is unstable especially for higher number of Zernike terms , and superpixel correction provides the best correction for our setup.

This method worked somewhat for low order aberrations but is unstable especially for higher order aberrations as the number of Zernike polynomials increases and increasing no of iterations. The authors mentioned the instability is due to the imperfect orthogonality of zernike modes. For our setup, we found that superpixel based aberration correction worked better.

I tried this method over top of superpixel correction and it did not improve the results. Since

the zernike coefficients are $< 0.2$ after superpixel correction, the perturbations are too small to measure any change in Strehl ratio. The authors also claimed that they were able to converge the coefficients till 0.2

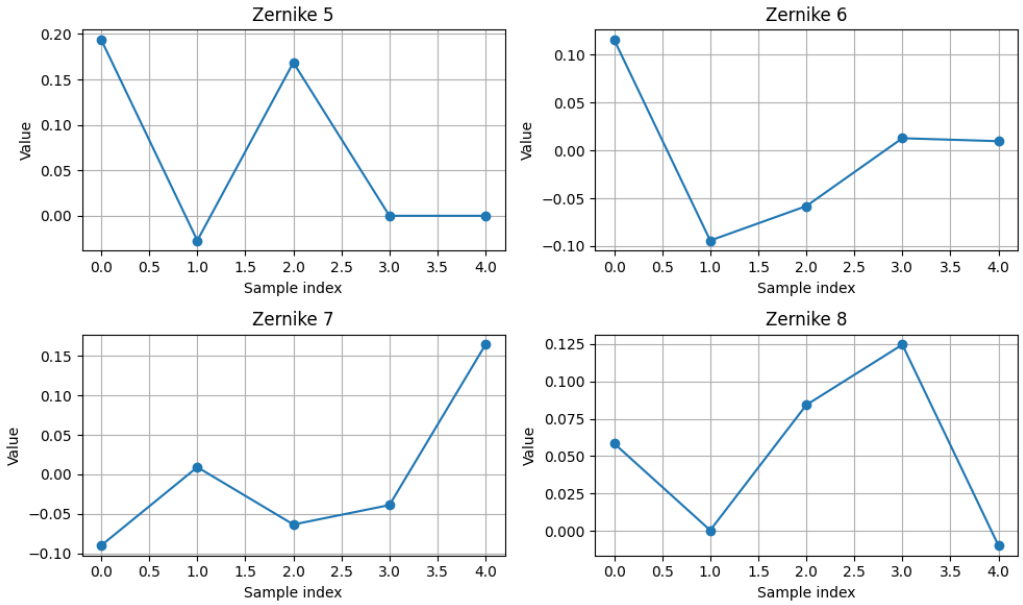



Figure 19: Evolution of selected Zernike coefficients across iterations during iterative Zernike perturbation over top of superpixel correction. Each subplot shows the change in coefficient values over iterations. We see improvement in the first iteration and subsequent iterations are unstable.

Other papers also mention about the inefficiency in convergence when optimising phases and other parameters together.

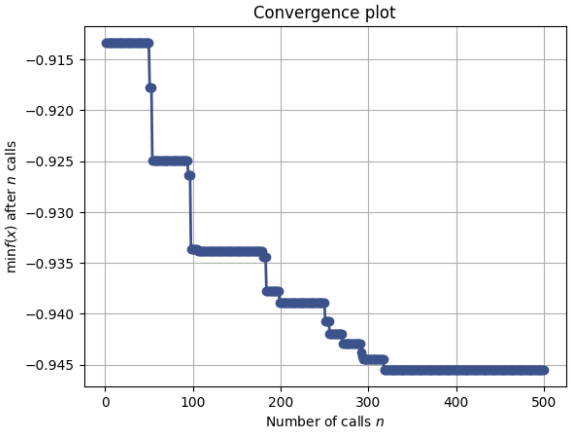
### 8.2.2 Bayesian Optimization in Zernike Kernel Space

Here, we use a multi armed bandit approach to optimize all the Zernike coefficients $w = \{w_n \mid n = 0, 1, 2, \ldots, n_{\max}\}$ at once to maximize the $R^2$ for a single bottle beam using Gaussian Processes and Upper Confidence Bound (UCB) as the acquisition function.

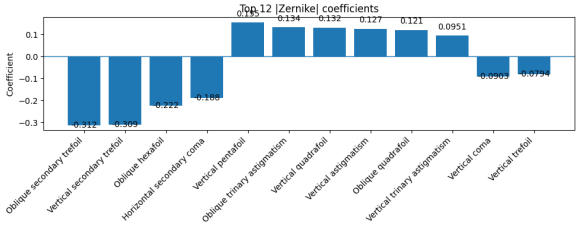
This is a powerful technique used in black box optimization problems where the objective function is unknown,expensive to evaluate and the evaluations are noisy. The UCB acquisition function balances exploration and exploitation by considering both the predicted mean and uncertainty of the unexplored space.

**Algorithm 3** Bayesian Optimization in Zernike Kernel Space

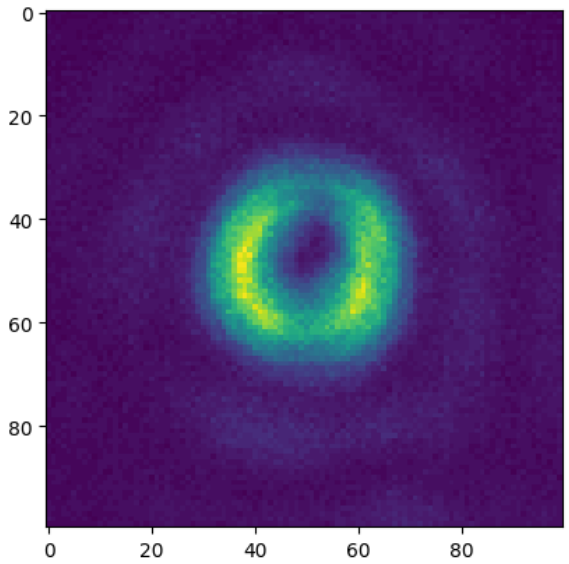
1: **Input:** Initial Zernike coefficients $w = \{w_n \mid n = 0, 1, 2, \ldots, n_{\max}\}$.
2: Step 1: Generate the phase mask $\phi_{\text{bottles}}$ using the initial Zernike coefficients.
3: Step 2: Measure the $R^2$ of the bottle beam and set it as the initial observation.
4: Step 3: Fit a Gaussian Process (GP) model to the observed data (Zernike coefficients and corresponding $R^2$ values).
5: Step 4: Use Upper Confidence Bound (UCB) as the acquisition function to select the next set of Zernike coefficients to evaluate.
6: Step 5: Generate the phase mask using the selected Zernike coefficients and measure the $R^2$.
7: Step 6: Update the GP model with the new observation.
8: Step 7: Repeat Steps 4-6 for a predefined number of iterations or until convergence.



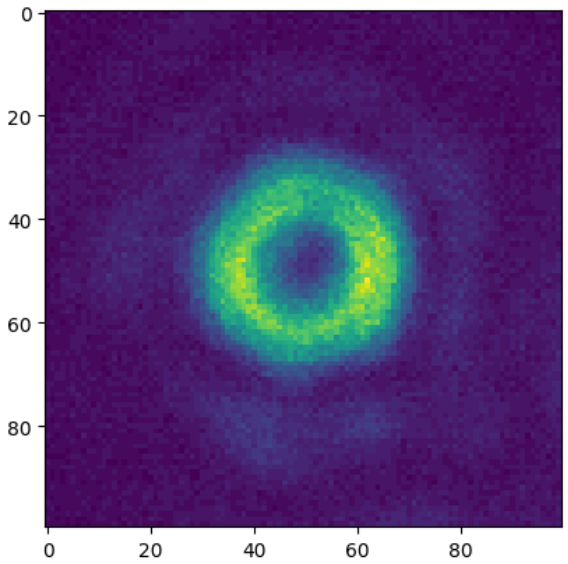

(a) Convergence plot




(b) Optimal Zernike coefficients

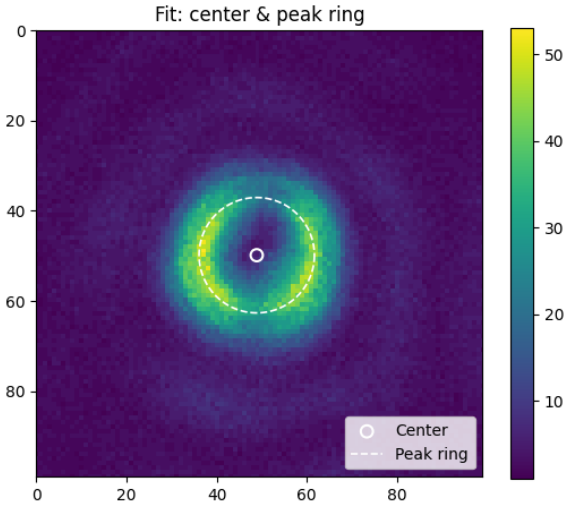


(c) Bottle beam before correction

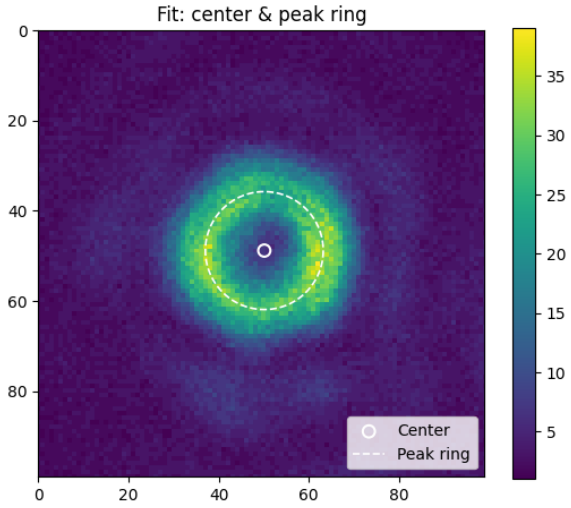


(d) Bottle beam after correction

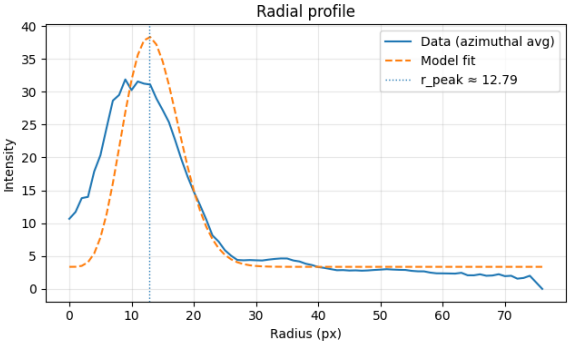
Figure 20: Results of aberration correction using GP-UCB on 50 Zernike polynomials. (a) Convergence of best $R^2$ over iterations. improvement of +3% (b) Optimal Zernike coefficients found. (c) Intensity profile of bottle beam before correction. (d) Intensity profile of bottle beam after correction showing improved uniformity and symmetry.



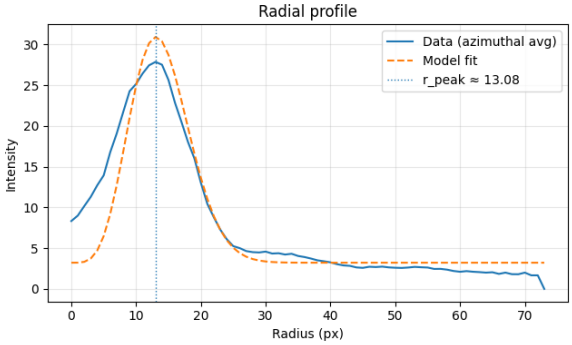

(a) Bottle beam fit (before correction).



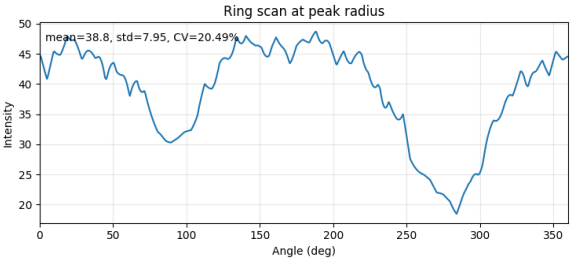

(b) Bottle beam fit (after correction).



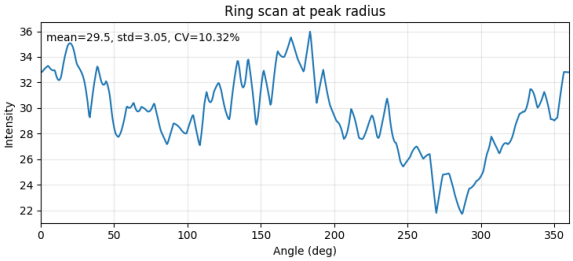

(c) Radial intensity profile (before correction).



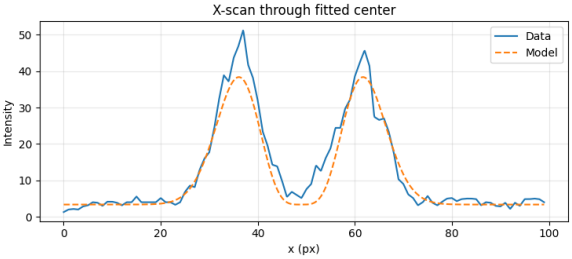

(d) Radial intensity profile (after correction).



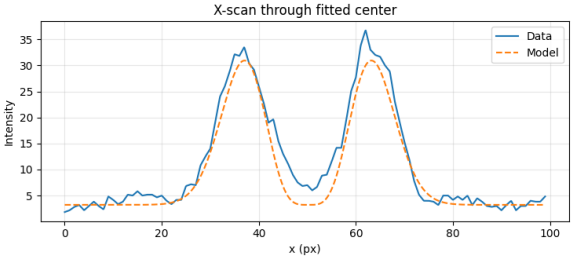

(e) Ring scan at peak radius (before).



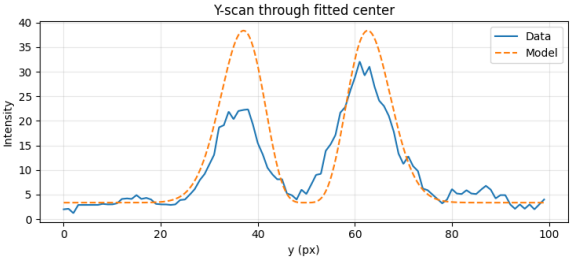

(f) Ring scan at peak radius (after).



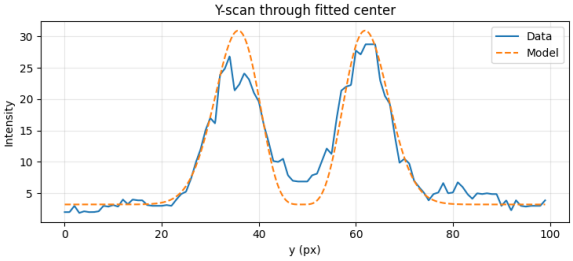

(g) X-scan through fitted center (before).




(h) X-scan through fitted center (after).




(i) Y-scan through fitted center (before).




(j) Y-scan through fitted center (after).



Figure 21: Comparison of fitted profiles and line scans before and after aberration correction. (a–b) Fitted bottle beams, (c–d) radial profiles, (e–f) ring scans, (g–h) x-scans, and (i–j) y-scans. Correction improves radial symmetry and uniformity of the bottle beam.

### 8.2.3 Code

The code to collect zernike perturbation data and getting metrics and the compressed zernike kernel holography is in src/zernike.py. The accompanying notebook src/notebooks/zernike.ipynb demonstrates the zernike calibration. SLM-Suite also has the zernike calibration code , our version is in pytorch and faster and more modular. The code for Bayesian optimization in zernike kernel space is demonstrated in src/notebooks/bayesianopt.ipynb

## 9 Quadratic Bottle Beams

Unlike tweezers the bottle beams have a quartic intensity profile instead of a quadratic profile. Here we discuss why this occurs and two ways we could mitigate this.

Consider Pi Phase mask to generate bottle beam.

$$\Phi(r) = \begin{cases} \pi, & r < R_0, \\ 0, & r \geq R_0, \end{cases}$$

where $R_0$ is the phase-step (cutoff) radius. For null intensity at center

$$R_0 \;=\; w_{\text{in}} \sqrt{\ln\!\Big(\tfrac{2}{1+e^{-H^2/w_{\text{in}}^2}}\Big)}.$$

The far field Amplitude is given by

$$A_{bb}(k,z) = \int_0^{R_0} r\, e^{-r^2\left(\frac{1}{w_{\text{in}}^2}+\frac{z}{2f}\right)} J_0(kr)\, dr \;-\; \int_{R_0}^{H} r\, e^{-r^2\left(\frac{1}{w_{\text{in}}^2}+\frac{z}{2f}\right)} J_0(kr)\, dr$$

The axial intensity is

$$I_{bb}(0,z) \approx \;\frac{16\,(\ln 2)^2}{w^8\, f^2}\, z^2 \;-\; \frac{16\,(\ln 2)^2((\ln 2)^2+12)}{3\,w^{12}\, f^4}\, z^4 \;+\; O(z^6).$$

while the radial Intensity is approximately $I_{bb}(r) \approx \left(r^2 e^{-r^2/w^2}\right)^2 \;\approx\; r^4 - \frac{2r^6}{w^2} + \frac{2r^8}{w^4} - \cdots$

Thus, the axial intensity is quadratic but radial intensity is quartic. This makes it difficult to trap atoms in a harmonic potential using bottle beams as discussed in [29].

We could add a quadratic by superposing an optical tweezer with the bottle beam

$$A_{\text{tot}}(r) \;=\; A_{bb}(r) \;+\; a\, e^{-r^2/w^2} \;=\; (r^2+a)\, e^{-r^2/w^2}\,.$$

Accordingly, the total intensity is

$$I_{\text{tot}}(r) = \left(r^2+a\right)^2 e^{-2r^2/w^2} \;\approx\; a^2 \;+\; \left(2a - \tfrac{2a^2}{w^2}\right) r^2 \;+\; \left(1 - \tfrac{4a}{w^2} + \tfrac{2a^2}{w^4}\right) r^4 \;+\; O(r^6).$$

$$I_{\text{tot, norm}}(r) = \frac{I_{\text{tot}}(r)}{\frac{w^2}{8}\left[(w^2+a)^2+a^2\right]} \;\approx\; \underbrace{\frac{8\,a^2}{w^2\left[(w^2+a)^2+a^2\right]}}_{r^0} + \underbrace{\frac{8\,(2aw^2-2a^2)}{w^4\left[(w^2+a)^2+a^2\right]}}_{r^2} r^2 + \underbrace{\frac{8\,(w^4-4aw^2+2a^2)}{w^6\left[(w^2+a)^2+a^2\right]}}_{r^4} r^4$$

We see a non zero quadratic term in the intensity expansion. We also see a non zero constant term which means we don't have null intensity at the center anymore. We could simulate the optimal superposition factor $a$ to get the best harmonicity in the potential as shown in Figure 22.

.

Another way to introduce quadraticity is to modify the pi phase mask by adjusting the

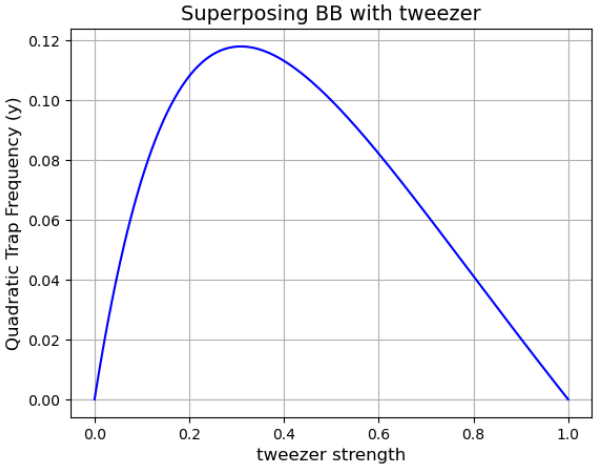



Figure 22: A simulation of superposed tweezer vs quadratic term strength of bottle beam

cutoff radius $R_0$ . This would introduce a quadratic profile in the far field amplitude but this also comes at the cost of having non zero intensity at the center of the bottle beam. We could simulate the optimal $R_0$ to get the best harmonicity in the potential as shown in Figure 23.

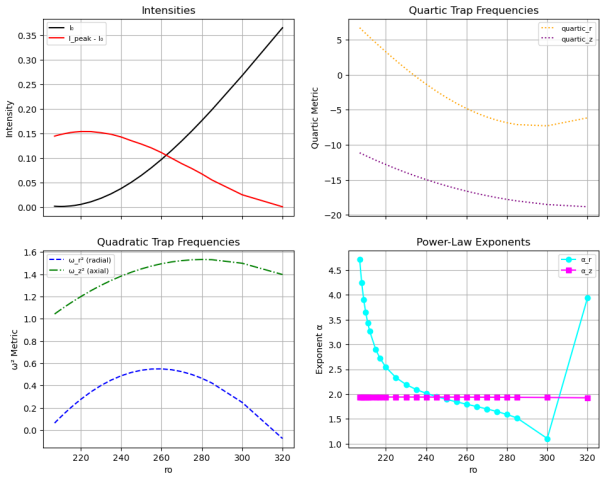



Figure 23: Simulation of $\pi$-mask cutoff radius vs. center intensity (black), escape intensity (red), quartic coefficients: radial (yellow) and axial (violet), quadratic coefficients: radial (blue) and axial (green), and $\alpha$ from fitting $r^\alpha$ to the bottle beam.

### 9.0.1 Code

The code to simulate the quadratic bottle beams is implemented in src/reports/quadraticbb.ipynb.

# 10 Outlook and Future Improvements

## 10.1 Tweezers

For the UltraPrecise tweezers in correcting the uniformities of tweezer arrays, The tweezer location detection needs to be more robust especially for large arrays and lower spacing between tweezers. We could use AI models like T-rex2 or grounding dino variants which has sota object detection accuracy to detect them.

## 10.2 Aberration Correction

Main source of aberration is from the physical setup which is cramped and little flexibility to adjust the optics. We did some hacks with the beam splitter divergence factor to compensate for the limited 4f lens focal length to focus the beam properly on the camera.

The Zernike polynomials didn't correlate well with the physical aberrations in the system which could be the reason for them to not work as expected. This implies that the primary source of aberration is not from zernike and which should be corrected before using zernike's. The bayesian optimisation showed promising results for correcting aberrations for bottle beams. We could try it once the physical setup is optimised. There's also potential in checking the pixel level optimisation for aberration correction . We could learn the perturbation functional from the ideal fourier transform using Fourier Neural operators which showed promising results in physics data and use this in designing the phase masks similar to whats done in [13].

## 10.3 3d optimisation

The 3d gradient methods is another direction to explore . Here there is an issue with computational resources. Especially in simulation for diffraction limited traps the discrete fourier transform doesn't provide enough far field resolution. We could use DTFT though its computationally expensive. Alternatives here are NUDFT or Non uniform Discrete FFTs which could have higher resolution near the bottle beam region and lower resolution elsewhere to save computation. Though the main blocker in using 3d gradient methods is the lack of a good 3d target profile for bottle beams especially for the modulate bottle beams. In 3d there are more constraints are some target profiles may not have a solution.

# 11 Lab Manual (Code Usage)

## 11.1 Repository and branch structure

All code lives in a single GitHub repository `holography`. The project is organized into two branches:

- `master`: lab workflow (SLM/camera pipeline, calibrations, tweezer/bottle-beam generation, and analysis notebooks).
- `simulation`: simulation workflow (OT/QBB experiments, gradient-based phase optimization, and simulation notebooks).

When reproducing results, always run from the correct branch because both branches contain a Python package named `src/` and the notebooks assume that local `src/`.

## 11.2 Recommended workflow

1. Use `master` for any lab-facing workflow and analysis.
2. Use `simulation` for simulation-only experiments, OT/QBB notebooks, and gradient-based phase optimization.
3. Please check both branches to find the code mentioned in the previous sections of the report (both branches have `src/`).

## 11.3 Local setup

### 11.3.1 Clone and checkout

```
git clone https://github.com/legendarybladerofmilkywaygalaxy/holography.git
cd holography

# Lab code
git checkout master

# Simulation code
git checkout simulation
```

### 11.3.2 Python dependencies

The repo uses `slmsuite` plus standard scientific Python packages. All the packages are installed in the lab computer. For your personal device, a minimal installation that works for most notebooks is:

```
pip install numpy scipy matplotlib pillow pandas scikit-image lmfit torch
pip install slmsuite
```

For OT/QBB notebooks (simulation branch only), you may additionally need:

```
pip install pot ott-jax jax jaxlib
```

### 11.3.3 Path convention used by the repo

Both branches use a `src/config.py` that sets `base_path` using the current working directory (e.g., `base_path = Path().resolve().parent`). Therefore:

- Run notebooks from inside the repository to keep paths consistent.
- If outputs are being written to an unexpected location, set `base_path` explicitly in `src/config.py`.

## 11.4 Module map (what each file is for)

This is the mental model of the code structure.

### 11.4.1 `master` branch (lab workflow)

- `src/slm_initialise.py`: defines `slm_initialise(...)` and constructs the core `fs` handle used throughout.
- `src/utils.py`: high-level utilities (camera capture, saving, calibration wrappers, tweezer generation, bottle-beam helpers).
- `src/zernike.py`: Zernike scan utilities and saving/loading of scan data.
- `src/plots.py`: fitting/plotting utilities used in analysis.
- `src/algos.py`: algorithmic routines used by notebooks (e.g., GS variants and helper pipelines).
- `src/gradient.py`: PyTorch gradient-based phase optimization models used in gradient experiments.

### 11.4.2 `simulation` branch (simulation workflow)

- `src/gradient.py`: simulation-side phase retrieval / optimization (2D and 3D variants) and target builders.
- `src/utils.py`: simulation helpers for generating targets and evaluating results.
- `notebooks/`: OT/QBB, bottle-beam, gradient, and report notebooks that reproduce simulation plots/results.
- `SLMTools/`: a Julia project (with `Project.toml`) containing phase retrieval / OT utilities used by some simulation workflows.

## 11.5 Minimal run recipes (copy/paste)

These are the minimal patterns used by the notebooks.

### 11.5.1 (A) Lab workflow (`master`)

**1) Initialize the core handle `fs`.**

```
from src.slm_initialise import slm_initialise
fs = slm_initialise(force=False, verbose=False)
```

**2) Acquire, plot, and save a camera frame.**

```
from src.utils import get_cam, plot_cam, save_cam

img  = get_cam(fs, exposure=1)
plot_cam(fs, exposure=1)
save_cam(fs, exposure=1, file_name="run1")
```

**3) Run or load calibrations.**

```
from src.utils import calibrate
calibrate(fs, vendor=True, refresh=True)   # refresh=False loads existing calibration(s)
```

### 11.5.2 Tweezer array generation (using our GS / UltraPrecise code)

Tweezers are generated using the custom routines in `src/algos.py`:

- `gs_spots(...)`: Gerchberg–Saxton (simulation) to obtain an initial near-field phase.
- `ultraprecise_spots(...)`: GS initialization + iterative spot correction ("UltraPrecise").

**Inputs.** The spot list must be a NumPy array of shape $(N, 2)$ in camera $(i, j)$ coordinates:

```
import numpy as np

# N x 2, each row = [i, j] in camera coordinates
spot_vectors = np.array([
    [900+100, 100],
    [900+100, 200],
    # ...
])
```

**Blaze phase (beam steering).** The UltraPrecise correction code expects a `blaze_phase` that is added to the final phase written to the SLM. This is generated using `slmsuite` phase utilities:

```
from slmsuite.holography.toolbox.phase import blaze
blaze_phase = blaze(fs, vector=(-0.003, -0.003))  # example; use the vector you use in the l
```

**(A) GS-only tweezers (quick baseline).**

```
from src.algos import gs_spots

phi_nf, phi_ff = gs_spots(spot_vectors, fs, n_iters=100)

# Write to SLM (typical usage)
fs.slm.write(phi_nf + blaze_phase, settle=True)
```

**(B) UltraPrecise tweezers (recommended for final uniformity/accuracy).** This runs GS to obtain a fixed far-field phase and then performs closed-loop correction iterations.

```
from src.algos import ultraprecise_spots

phis = ultraprecise_spots(
    spot_vectors, fs, blaze_phase,
    n_iters_gs=100,
    n_iters_corr=100
)

phi_nf_final = phis[-1]
fs.slm.write(phi_nf_final + blaze_phase, settle=True)
```

**Notes on outputs.** `ultraprecise_spots` returns a list `phis` containing the near-field phase estimate at each correction iteration. The final phase mask is typically taken as `phis[-1]`.

**Where this is executed.** The end-to-end workflow (spot list → GS → UltraPrecise refinement → camera evaluation) is run in `notebooks/tweezer_array.ipynb`.

**5) Bottle-beam phase and axial scan.**

```
from src.utils import bottle_beam_phase, get_3d_images, plot_axial

phi = bottle_beam_phase(fs)
stack = get_3d_images(fs, phase=phi, N=100)
plot_axial(plane="XZ", index=50, images=stack)
```

### 11.5.3 (B) Simulation workflow (`simulation`)

The simulation notebooks are the recommended entry point. Key utilities include:

- CG-based phase retrieval modules for 2D and 3D bottle-beam targets (in `src/gradient.py`).
- OT/QBB experiments in `notebooks/ot.ipynb` and `notebooks/qbb.ipynb`.

Typical simulation usage is:

1. build a target intensity / volume,
2. optimize a phase mask with a CG/gradient routine,
3. evaluate metrics (line scans, uniformity, Strehl-like metrics) using notebook utilities.

### 11.6 Practical notes

- Treat notebooks as the "source of truth" for reproducible workflows; the Python modules provide reusable building blocks.
- There are some dev notebooks which have a lot of code in testing. These are not maintained and may be broken.
- Go through this report and the `slmsuite` notebooks to get a general idea of how the algorithms work.

## 12 References

## References


[1] D. Petrosyan et al. “Fast measurements and multiqubit gates in dual-species atomic arrays”. In: *Phys. Rev. A* 110 (4 Oct. 2024), p. 042404. URL: https://link.aps.org/doi/10.1103/PhysRevA.110.042404.

[2] Kurt D. Wulff et al. “Aberration correction in holographic optical tweezers”. In: *Opt. Express* 14.9 (May 2006), pp. 4169–4174. URL: https://opg.optica.org/oe/abstract.cfm?URI=oe-14-9-4169.

[3] Ralph W Gerchberg. “A practical algorithm for the determination of plane from image and diffraction pictures”. In: *Optik* 35.2 (1972), pp. 237–246.

[4] Roberto Di Leonardo, Francesca Ianni, and Giancarlo Ruocco. “Computer generation of optimal holograms for optical trap arrays”. In: *Opt. Express* 15.4 (Feb. 2007), pp. 1913–1922. URL: https://opg.optica.org/oe/abstract.cfm?URI=oe-15-4-1913.

[5] Hyosub Kim et al. “Gerchberg-Saxton algorithm for fast and efficient atom rearrangement in optical tweezer traps”. In: *Opt. Express* 27.3 (Feb. 2019), pp. 2184–2196. URL: https://opg.optica.org/oe/abstract.cfm?URI=oe-27-3-2184.

[6] Y. T. Chew et al. “Ultraprecise holographic optical tweezer array”. In: *Physical Review A* 110.5 (Nov. 2024). ISSN: 2469-9934. URL: http://dx.doi.org/10.1103/PhysRevA.110.053518.

[7] P. Memmolo et al. “Investigation on specific solutions of Gerchberg–Saxton algorithm”. In: *Optics and Lasers in Engineering* 52 (Jan. 2014), pp. 206–211.

[8] Matthew Pasienski and Brian DeMarco. “A high-accuracy algorithm for designing arbitrary holographic atom traps”. In: *Opt. Express* 16.3 (Feb. 2008), pp. 2176–2190. URL: https://opg.optica.org/oe/abstract.cfm?URI=oe-16-3-2176.

[9] Rohan Chandra et al. *PhasePack: A Phase Retrieval Library*. 2017. arXiv: 1711.10175 [math.OC]. URL: https://arxiv.org/abs/1711.10175.

[10] Xuejin Sui, Zhe He, Daping Chu, et al. “Non-convex optimization for inverse problem solving in computer-generated holography”. In: *Light: Science & Applications* 13 (2024), p. 158. URL: https://doi.org/10.1038/s41377-024-01446-w.

[11] Jingzhao Zhang et al. “3D computer-generated holography by non-convex optimization”. In: *Optica* 4.10 (Oct. 2017), pp. 1306–1313. URL: https://opg.optica.org/optica/abstract.cfm?URI=optica-4-10-1306.

[12] Tiffany Harte et al. “Conjugate gradient minimisation approach to generating holographic traps for ultracold atoms”. In: *Opt. Express* 22.22 (Nov. 2014), pp. 26548–26558. URL: https://opg.optica.org/oe/abstract.cfm?URI=oe-22-22-26548.

[13] Yifan Peng et al. “Neural Holography with Camera-in-the-loop Training”. In: *ACM Transactions on Graphics (TOG)* 39.6 (2020). Presented at SIGGRAPH Asia 2020, 185:1–185:14.

[14] Han-Ju Yeom, Keehoon Hong, and Minsik Park. "High-quality phase-only Fourier hologram generation with camera-in-the-loop". In: *Opt. Express* 33.4 (Feb. 2025), pp. 6615–6628. URL: https://opg.optica.org/oe/abstract.cfm?URI=oe-33-4-6615.

[15] Antonio Ciarlo et al. *Deep Learning for Optical Tweezers*. 2024. arXiv: 2401.02321 [physics.optics]. URL: https://arxiv.org/abs/2401.02321.

[16] Jing Zhao et al. "Design of Optical Tweezers Manipulation Control System Based on Novel Self-Organizing Fuzzy Cerebellar Model Neural Network". In: *Applied Sciences* 12.19 (2022). ISSN: 2076-3417. URL: https://www.mdpi.com/2076-3417/12/19/9655.

[17] Andreas Erik Gejl Madsen, René Lynge Eriksen, and Jesper Glückstad. "Comparison of state-of-the-art Computer Generated Holography algorithms and a machine learning approach". In: *Optics Communications* 505 (2022), p. 127590. ISSN: 0030-4018. URL: https://www.sciencedirect.com/science/article/pii/S0030401821008397.

[18] Siyuan Zhang. "Trapping and Rydberg Excitation of a Single Atom Qubit in a Blue Detuned Bottle Beam". Accessed: April 2025. Ph.D. Dissertation. Madison, Wisconsin: University of Wisconsin–Madison, 2012. URL: https://digital.library.wisc.edu/1711.dl/XFYY5KUN2UWBI9E.

[19] Peng Xu et al. "Trapping a single atom in a blue detuned optical bottle beam trap". In: *Opt. Lett.* 35.13 (July 2010), pp. 2164–2166. URL: https://opg.optica.org/ol/abstract.cfm?URI=ol-35-13-2164.

[20] Yuzhe Xiao et al. "Efficient generation of optical bottle beams". In: *Nanophotonics* 10.11 (July 2021), pp. 2893–2901. ISSN: 2192-8614. URL: http://dx.doi.org/10.1515/nanoph-2021-0243.

[21] Jatin Abacousnac and David G. Grier. "Dexterous holographic trapping of dark-seeking particles with Zernike holograms". In: *Optics Express* 30.13 (June 2022), p. 23568. ISSN: 1094-4087. URL: http://dx.doi.org/10.1364/OE.458544.

[22] Peng Zhang et al. "Generation of acoustic self-bending and bottle beams by phase engineering". In: *Nature Communications* 5 (2014), p. 4316. URL: https://www.nature.com/articles/ncomms5316.

[23] Leopold Konzett. "Array of Optical Bottle Beams Using a Liquid Crystal Spatial Light Modulator". Accessed: April 2025. Master's Thesis. ETH Zürich, 2021. URL: https://ethz.ch/content/dam/ethz/special-interest/phys/quantum-electronics/tiqi-dam/documents/masters_theses/Masterthesis_Leopold_Konzett.

[24] Jeffrey A. Davis and Don M. Cottrell. "Random mask encoding of multiplexed phase-only and binary phase-only filters". In: *Opt. Lett.* 19.7 (Apr. 1994), pp. 496–498. URL: https://opg.optica.org/ol/abstract.cfm?URI=ol-19-7-496.

[25] Nicolas Perez and Daryl Preece. "Enhanced optical vector bottle beams with obscured nodal surfaces". In: *Opt. Express* 32.8 (Apr. 2024), pp. 14010–14017. URL: https://opg.optica.org/oe/abstract.cfm?URI=oe-32-8-14010.

[26] Hunter Swan et al. “High-fidelity holographic beam shaping with optimal transport and phase diversity”. In: *Opt. Express* 33.3 (Feb. 2025), pp. 6290–6303. URL: https://opg.optica.org/oe/abstract.cfm?URI=oe-33-3-6290.

[27] Tomáš Čižmár, Michael Mazilu, and Kishan Dholakia. “In situ wavefront correction and its application to micromanipulation”. In: *Nature Photonics* 4.6 (2010), pp. 388–394. URL: https://doi.org/10.1038/nphoton.2010.85.

[28] Ian Christen et al. *Full-volume aberration-space holography*. 2025. arXiv: 2505.08777 [physics.optics]. URL: https://arxiv.org/abs/2505.08777.

[29] P. Huft et al. “Simple, passive design for large optical trap arrays for single atoms”. In: *Phys. Rev. A* 105 (6 June 2022), p. 063111. URL: https://link.aps.org/doi/10.1103/PhysRevA.105.063111.

[30] Donggyu Kim et al. “Large-scale uniform optical focus array generation with a phase spatial light modulator”. In: *Optics Letters* 44.12 (June 2019), p. 3178. ISSN: 1539-4794. URL: http://dx.doi.org/10.1364/OL.44.003178.

[31] Mario Montes-Usategui et al. “Fast generation of holographic optical tweezers by random mask encoding of Fourier components”. In: *Opt. Express* 14.6 (Mar. 2006), pp. 2101–2107. URL: https://opg.optica.org/oe/abstract.cfm?URI=oe-14-6-2101.

[32] Zejian Ren et al. *Creation of a tweezer array for cold atoms utilizing a generative neural network*. 2024. arXiv: 2401.06014 [physics.optics]. URL: https://arxiv.org/abs/2401.06014.

[33] Paul Hill et al. “Optical phase aberration correction with an ultracold quantum gas”. In: *Phys. Rev. A* 110 (5 Nov. 2024), p. 053308. URL: https://link.aps.org/doi/10.1103/PhysRevA.110.053308.

[34] Yansheng Liang et al. “Aberration correction in holographic optical tweezers using a high-order optical vortex”. In: *Appl. Opt.* 57.13 (May 2018), pp. 3618–3623. URL: https://opg.optica.org/ao/abstract.cfm?URI=ao-57-13-3618.

[35] A A Pushkina et al. “Comprehensive model and performance optimization of phase-only spatial light modulators”. In: *Measurement Science and Technology* 31.12 (Oct. 2020), p. 125202. ISSN: 1361-6501. URL: http://dx.doi.org/10.1088/1361-6501/aba56b.

[36] Harshil Dave, Sean Krupa, and Paul Lebow. “Phase recovery from Fresnel incoherent correlation holography using differential Zernike fitting”. In: *Opt. Lett.* 49.18 (Sept. 2024), pp. 5023–5026. URL: https://opg.optica.org/ol/abstract.cfm?URI=ol-49-18-5023.

[37] Akondi Vyas et al. *Spatial Light Modulator for wavefront correction*. 2009. arXiv: 0909.3413 [astro-ph.IM]. URL: https://arxiv.org/abs/0909.3413.

[38] Alexander B Stilgoe and Halina Rubinsztein-Dunlop. “Wave characterisation and aberration correction using hybrid direct search”. In: *Journal of Optics* 23.8 (July 2021), p. 085602. URL: https://dx.doi.org/10.1088/2040-8986/ac094d.

[39] Andrzej Kaczorowski et al. “Optimization-Based Adaptive Optical Correction for Holographic Projectors”. In: *Journal of Display Technology* 11.7 (2015), pp. 596–603.

[40] Nikolai Suchkov et al. “Simultaneous aberration and aperture control using a single spatial light modulator”. In: *Opt. Express* 27.9 (Apr. 2019), pp. 12399–12413. URL: https://opg.optica.org/oe/abstract.cfm?URI=oe-27-9-12399.

[41] D. Barredo et al. “Three-Dimensional Trapping of Individual Rydberg Atoms in Ponderomotive Bottle Beam Traps”. In: *Phys. Rev. Lett.* 124 (2 Jan. 2020), p. 023201. URL: https://link.aps.org/doi/10.1103/PhysRevLett.124.023201.

[42] G. Li et al. “Crossed vortex bottle beam trap for single-atom qubits”. In: *Opt. Lett.* 37.5 (Mar. 2012), pp. 851–853. URL: https://opg.optica.org/ol/abstract.cfm?URI=ol-37-5-851.

[43] D. Petrosyan et al. *Fast measurements and multiqubit gates in dual species atomic arrays*. 2024. arXiv: 2406.07356 [quant-ph]. URL: https://arxiv.org/abs/2406.07356.

[44] Chengyu Fang et al. *Interleaved dual-species arrays of single atoms using a passive optical element and one trapping laser*. 2025. arXiv: 2412.20261 [physics.atom-ph]. URL: https://arxiv.org/abs/2412.20261.

[45] Sivan Trajtenberg Mills et al. “qp-SLM: An open-source holography suite for spatial light modulators”. In: *Conference on Lasers and Electro-Optics*. Optica Publishing Group, 2022, STu4F.4. URL: https://opg.optica.org/abstract.cfm?URI=CLEO_SI-2022-STu4F.4.

[46] Shraddha Anand et al. *A dual-species Rydberg array*. 2024. arXiv: 2401.10325 [quant-ph]. URL: https://arxiv.org/abs/2401.10325.

[47] Ian Christen et al. “An integrated photonic engine for programmable atomic control”. In: *Nature Communications* 16.1 (Jan. 2025). ISSN: 2041-1723. URL: http://dx.doi.org/10.1038/s41467-024-55423-3.